

University Of Asia Pacific

Department of Computer Science and Engineering

CSE 402: Operating System Lab

LAB REPORT

Report Number: 3

Report Title: FCFS, SJF (non-preemptive) and Round Robin CPU Scheduling Comparison

Submitted by:

Name: Md. Muhaimin Al Baki Raiyan

Student ID: 23101163

Section: C2

Submitted To:

Atia Rahman Orthi

Lecturer,

Department of Computer Science and Engineering

Date of Submission: 17.08.2026

CSE 402 – CPU Scheduling Comparative Report

1. Introduction

CPU scheduling is the mechanism by which an operating system decides which process in the ready queue gets access to the CPU at any given time. Because the CPU is a single shared resource and many processes compete for it, the scheduling policy directly determines how efficiently the system utilizes its resources and how fairly and quickly individual processes are served. This report implements and compares three classic CPU scheduling algorithms — First-Come, First-Served (FCFS), Shortest Job First (SJF, non-preemptive), and Round Robin (RR) — using a Python simulation, and evaluates their performance using two standard metrics: Turnaround Time (TAT) and Waiting Time (WT).

2. Objective

- To simulate FCFS, SJF (non-preemptive), and Round Robin CPU scheduling algorithms programmatically.
- To construct the Gantt chart for each algorithm and compute Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for every process.
- To compare the three algorithms on the basis of average TAT and average WT for an identical process set.
- To analyze the trade-offs between the algorithms and identify which one performs best under the given workload.

3. Process Set (Input Data)

The following set of six processes, with arrival time (AT) and burst time (BT) given in milliseconds, was used as the common input for all three algorithms. For Round Robin, a time quantum of 5s was used.

Process	Arrival Time	Burst Time
P1	0	7
P2	1	4
P3	2	15
P4	3	11
P5	4	20
P6	4	9

4. Algorithm Implementations and Results

4.1 First-Come, First-Served (FCFS)

FCFS schedules processes strictly in the order of their arrival, with no preemption. Once a process starts execution, it runs to completion before the next process (in arrival order) is dispatched. It is implemented by sorting the process list by arrival time and executing each process sequentially, inserting an IDLE block whenever the CPU has no process to run.

P1(0-7) | P2(7-11) | P3(11-26) | P4(26-37) | P5(37-57) | P6(57-66)

PID	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	1	4	11	10	6
P3	2	15	26	24	9
P4	3	11	37	34	23
P5	4	20	57	53	33
P6	4	9	66	62	53

Average Turnaround Time (TAT): 31.67 ms **Average Waiting Time (WT):** 20.67 ms

4.2 Shortest Job First – Non-Preemptive (SJF)

Non-preemptive SJF always selects, from the set of processes that have already arrived, the one with the smallest burst time to run next; once selected, that process runs to completion without interruption. This requires re-evaluating the ready queue every time the CPU becomes free, which the implementation does by scanning all not-yet-completed processes whose arrival time is less than or equal to the current time and picking the minimum burst time among them.

P1(0-7) | P2(7-11) | P6(11-20) | P4(20-31) | P3(31-46) | P5(46-66)

PID	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	1	4	11	10	6
P3	2	15	46	44	29
P4	3	11	31	28	17
P5	4	20	66	62	42
P6	4	9	20	16	7

Average Turnaround Time (TAT): 27.83 ms **Average Waiting Time (WT):** 16.83 ms

4.3 Round Robin (Quantum = 5s)

Round Robin gives every process a fixed time slice, or quantum (5s here), in a circular ready queue. If a process does not finish within its quantum, it is preempted and placed at the back of the queue, and newly arrived processes are appended to the queue as they arrive. This continues until all processes complete, producing more context switches than FCFS or SJF but guaranteeing that no process waits indefinitely.

P1(0-5) | P2(5-9) | P3(9-14) | P4(14-19) | P5(19-24) | P6(24-29) | P1(29-31) | P3(31-36) | P4(36-41) | P5(41-46) | P6(46-50) | P3(50-55) | P4(55-56) | P5(56-61) | P5(61-66)

PID	AT	BT	CT	TAT	WT
P1	0	7	31	31	24
P2	1	4	9	8	4
P3	2	15	55	53	38
P4	3	11	56	53	42
P5	4	20	66	62	42
P6	4	9	50	46	37

Average Turnaround Time (TAT): 42.17 ms **Average Waiting Time (WT):** 31.17 ms

5. Comparative Summary

Algorithm	Average TAT (ms)	Average WT (ms)
FCFS	31.67	20.67
SJF (Non-Preemptive)	27.83	16.83
Round Robin (Q = 5)	42.17	31.17

Lowest Average TAT: SJF (27.83 ms) **Lowest Average WT:** SJF (16.83 ms)

6. Discussion

The results reveal clear and expected performance differences among the three algorithms, rooted in how each one orders CPU access.

FCFS produced an average TAT of 31.67 ms and average WT of 20.67 ms. Its weakness is directly visible in the Gantt chart: P3 (burst time 15) and P5 (burst time 20) run early simply because they arrived earlier, forcing every process behind them — including short ones — to wait far longer than their own execution time. This is the classic convoy effect, where a handful of long processes at the front of the queue block many shorter processes behind them, inflating average waiting time even though no process is ever preempted or starved indefinitely. FCFS is simple to implement and completely fair in the sense of first-arrived-first-served, but it is oblivious to job length, which is precisely why it performs worst in this comparison on both metrics.

SJF (non-preemptive) achieved the best results overall, with average TAT of 27.83 ms and average WT of 16.83 ms — noticeably better than both other algorithms. By always choosing the shortest available job next, SJF minimizes the number of processes forced to wait behind long ones; in this run it dispatched P6 (BT=9) ahead of P3 (BT=15), even though P3 arrived earlier, which is exactly the reordering that reduces average waiting time. This matches the theoretical guarantee that SJF is provably optimal for minimizing average waiting time among non-preemptive algorithms, given a fixed set of ready processes. Its real-world limitation, not visible in this particular result set, is that it requires advance knowledge of burst time (rarely available in practice) and can starve long processes if short jobs keep arriving — neither of which affected this particular six-process workload since all processes arrived within the first 4 ms.

Round Robin performed worst here, with average TAT of 42.17 ms and average WT of 31.17 ms — noticeably worse than both FCFS and SJF. This is a direct consequence of the quantum (5s) being small relative to the burst times of most processes (four of the six processes have burst times of 9 ms or more). Long processes such as P3, P4, and P5 must be repeatedly preempted and re-queued, so their completion is spread out over many rounds; for example, P5 (burst time 20) does not finish until time 66, three separate turns after first being dispatched. The Gantt chart shows fifteen separate execution slices for only six processes, versus six slices for FCFS and SJF — a clear sign of scheduling overhead. Round Robin's real strength (better response time and fairness for interactive workloads) does not show up in these particular metrics, because TAT and WT reward jobs finishing sooner overall, while RR intentionally trades that off for giving every process a turn early and often.

Taken together, the comparison illustrates a fundamental scheduling trade-off: policies that use burst-time information (SJF) can substantially reduce average waiting time, policies that ignore process length entirely (FCFS) are simple but vulnerable to the convoy effect, and quantum-based preemptive policies (Round Robin) sacrifice average TAT/WT in exchange for responsiveness and starvation-freedom, with performance highly sensitive to the choice of quantum. A quantum much larger than most burst times

would make Round Robin behave closer to FCFS, while a quantum much smaller than burst times increases context-switch overhead and average waiting time, as observed with Q = 5 in this workload.

7. Conclusion

This lab exercise implemented and empirically compared three CPU scheduling algorithms — FCFS, non-preemptive SJF, and Round Robin — on an identical set of six processes with varying arrival and burst times. The simulation confirms core operating systems theory: SJF minimized both average turnaround time (27.83 ms) and average waiting time (16.83 ms) because it consistently prioritizes shorter jobs, making it the most efficient policy for this workload purely in terms of these two metrics. FCFS was measurably worse (31.67 ms TAT, 20.67 ms WT) due to the convoy effect caused by long processes arriving early, while Round Robin with a 5s quantum performed the worst of the three (42.17 ms TAT, 31.17 ms WT) because the mismatch between the quantum size and the generally large burst times in this process set led to excessive preemption and re-queuing.

No single algorithm is universally superior — the ranking obtained here (SJF best, FCFS second, Round Robin last) is specific to this workload and these metrics. SJF's advantage would shrink or reverse under a workload with many similarly-sized processes, and it is rarely usable in practice because it requires burst times to be known in advance. Round Robin, despite scoring worst on average TAT/WT in this run, would outperform both FCFS and SJF on response-time-sensitive metrics for interactive or time-sharing systems, and its performance would improve substantially with a better-tuned quantum (for instance, closer to the average burst time of about 11 ms rather than 5s). In practice, real operating systems (e.g., multilevel feedback queues) combine ideas from all three approaches — approximating SJF's efficiency, Round Robin's fairness and responsiveness, and FCFS's simplicity — rather than relying on any single pure algorithm.

8. Python Source Code

The three scheduling algorithms, along with the Gantt chart construction, metric calculation, and comparison routines, were implemented in Python as shown below.

```
processes = [
    {'pid': 'P1', 'at': 0, 'bt': 7},
    {'pid': 'P2', 'at': 1, 'bt': 4},
    {'pid': 'P3', 'at': 2, 'bt': 15},
    {'pid': 'P4', 'at': 3, 'bt': 11},
    {'pid': 'P5', 'at': 4, 'bt': 20},
    {'pid': 'P6', 'at': 4, 'bt': 9},
]

QUANTUM = 5


def fcfs(processes):
    plist = sorted(processes, key=lambda p: p['at'])
    completed = []
    gantt = []
    time = 0

    for p in plist:
        if time < p['at']:
            gantt.append(('IDLE', time, p['at']))
            time = p['at']

        start = time
        end = time + p['bt']
        gantt.append((p['pid'], start, end))
        time = end

        ct = end
        tat = ct - p['at']
        wt = tat - p['bt']

        completed.append({
            'pid': p['pid'], 'at': p['at'], 'bt': p['bt'],
            'ct': ct, 'tat': tat, 'wt': wt
        })

    return completed, gantt


def sjf_non_preemptive(processes):
    plist = sorted(processes, key=lambda p: p['at'])
    not_done = list(plist)
    completed = []
    gantt = []
    time = 0
```

```
    while not_done:
        ready = []
        for p in not_done:
            if p['at'] <= time:
                ready.append(p)

        if not ready:
            next_arrival = min(p['at'] for p in not_done)
            gantt.append(('IDLE', time, next_arrival))
            time = next_arrival
            continue

        shortest = ready[0]
        for p in ready:
            if p['bt'] < shortest['bt']:
                shortest = p

        not_done.remove(shortest)

        start = time
        end = time + shortest['bt']
        gantt.append((shortest['pid'], start, end))
        time = end

        ct = end
        tat = ct - shortest['at']
        wt = tat - shortest['bt']

        completed.append({
            'pid': shortest['pid'], 'at': shortest['at'], 'bt': shortest['bt'],
            'ct': ct, 'tat': tat, 'wt': wt
        })

    return completed, gantt


def round_robin(processes, quantum):
    plist = sorted(processes, key=lambda p: p['at'])

    remaining_time = {}
    for p in plist:
        remaining_time[p['pid']] = p['bt']

    gantt = []
    ready_queue = []
    finish_info = {}
    time = 0
    next_index = 0

    while next_index < len(plist) and plist[next_index]['at'] <= time:
        ready_queue.append(plist[next_index])
        next_index += 1
```

```
    if not ready_queue:
        time = plist[0]['at']
        while next_index < len(plist) and plist[next_index]['at'] <= time:
            ready_queue.append(plist[next_index])
            next_index += 1

    while ready_queue:
        p = ready_queue.pop(0)
        pid = p['pid']

        run_for = min(quantum, remaining_time[pid])
        start = time
        end = time + run_for
        gantt.append((pid, start, end))

        time = end
        remaining_time[pid] -= run_for

        while next_index < len(plist) and plist[next_index]['at'] <= time:
            ready_queue.append(plist[next_index])
            next_index += 1

        if remaining_time[pid] > 0:
            ready_queue.append(p)
        else:
            ct = time
            tat = ct - p['at']
            wt = tat - p['bt']
            finish_info[pid] = {'pid': pid, 'at': p['at'], 'bt': p['bt'], 'ct': ct,
'tat': tat, 'wt': wt}

        if not ready_queue and next_index < len(plist):
            if plist[next_index]['at'] > time:
                gantt.append(('IDLE', time, plist[next_index]['at']))
                time = plist[next_index]['at']
            while next_index < len(plist) and plist[next_index]['at'] <= time:
                ready_queue.append(plist[next_index])
                next_index += 1

    completed = [finish_info[p['pid']] for p in plist]
    return completed, gantt


def get_averages(completed):
    total_tat = 0
    total_wt = 0
    for p in completed:
        total_tat += p['tat']
        total_wt += p['wt']
    n = len(completed)
    return total_tat / n, total_wt / n
```

```
def show_results(title, completed, gantt):
    print(f"{title} ")
    print("Gantt Chart:")
    print(" | ".join(f"{pid}({s}-{e})" for pid, s, e in gantt))
    print()
    print(f"{'PID':<5}{'AT':<5}{'BT':<5}{'CT':<5}{'TAT':<5}{'WT':<5}")

    for p in sorted(completed, key=lambda x: x['pid']):
        print(f"{p['pid']:<5}{p['at']:<5}{p['bt']:<5}{p['ct']:<5}{p['tat']:<5}
{p['wt']:<5}")

    avg_tat, avg_wt = get_averages(completed)
    print(f"\nAverage TAT: {avg_tat:.2f}")
    print(f"Average WT: {avg_wt:.2f}\n")
    return avg_tat, avg_wt


fcfs_result, fcfs_gantt = fcfs(processes)
sjf_result, sjf_gantt = sjf_non_preemptive(processes)
rr_result, rr_gantt = round_robin(processes, QUANTUM)

fcfs_tat, fcfs_wt = show_results("FCFS", fcfs_result, fcfs_gantt)
sjf_tat, sjf_wt = show_results("SJF (Non-Preemptive)", sjf_result, sjf_gantt)
rr_tat, rr_wt = show_results(f"Round Robin (Quantum = {QUANTUM})", rr_result,
rr_gantt)

print(" Final Comparison ")
print(f"{'Algorithm':<25}{'Avg TAT':<12}{'Avg WT':<12}")
print(f"{'FCFS':<25}{fcfs_tat:<12.2f}{fcfs_wt:<12.2f}")
print(f"{'SJF':<25}{sjf_tat:<12.2f}{sjf_wt:<12.2f}")
print(f"{'Round Robin (Q=' + str(QUANTUM) + ')':<25}{rr_tat:<12.2f}{rr_wt:<12.2f}")

scores = {
    'FCFS': (fcfs_tat, fcfs_wt),
    'SJF': (sjf_tat, sjf_wt),
    'Round Robin': (rr_tat, rr_wt)
}
best_tat_algo = min(scores, key=lambda k: scores[k][0])
best_wt_algo = min(scores, key=lambda k: scores[k][1])

print(f"\nLowest Average TAT: {best_tat_algo} ({scores[best_tat_algo][0]:.2f})")
print(f"Lowest Average WT: {best_wt_algo} ({scores[best_wt_algo][1]:.2f})")
```